SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	A.Lakshmipathi	**Reg. No.:**	192472323
Section / Batch:		**Date:**	27-10-2026

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

REGULAR EXPRESSIONS – APPLICATION BASED QUESTIONS

1. Resume Information Extraction

Aim

To develop a Python program using Regular Expressions to extract candidate details from resume text, identify email addresses and mobile numbers, detect technical skills and years of experience, summarize the profile, and shortlist candidates who have at least 2 years of experience and Python skill.

Algorithm

1. Start the program and store the resume text in a string.
2. Use regular expressions to extract the candidate name, email address, mobile number, and years of experience.
3. Search the resume for technical skills such as Python, Java, SQL, Machine Learning, and NLP.
4. Create a structured summary of the extracted information.
5. Check whether experience is at least 2 years and Python is listed as a skill.
6. Display the extracted details and shortlist status; stop.

Python Code

```
import re

resume = """ Name:
Priya Kumar
Email: priya.kumar@example.com
Phone: 9876543210
Skills: Python, SQL, Machine Learning, NLP Experience: 3
years
"""

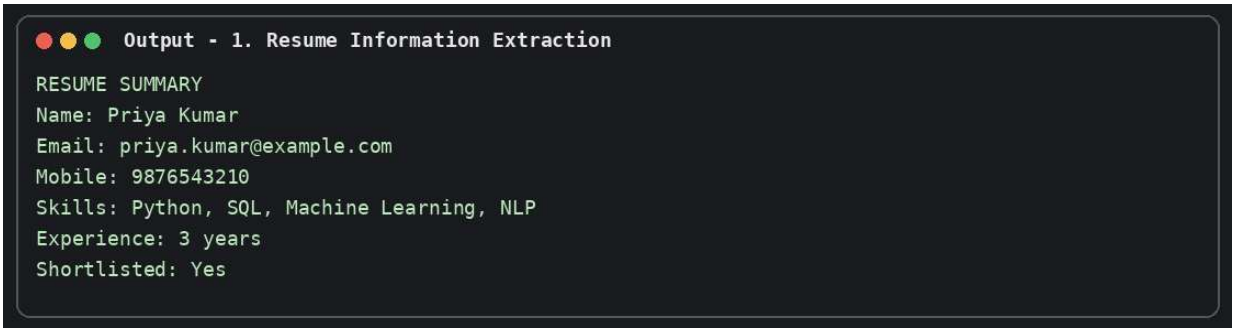
name = re.search(r"Name:\s*(.+)", resume)
email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume) phone
= re.search(r"\b[6-9]\d{9}\b", resume)
experience = re.search(r"Experience:\s*(\d+)\s*years?", resume)

skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = [skill for skill in skill_list
        if re.search(r"\b" + re.escape(skill) + r"\b", resume,
                re.IGNORECASE)]

years = int(experience.group(1)) if experience else 0 candidate_name =
name.group(1).strip() if name else "Not found" candidate_email =
email.group(0) if email else "Not found" candidate_phone =
phone.group(0) if phone else "Not found"
```

```
print("RESUME SUMMARY")
print("Name:", candidate_name)
print("Email:", candidate_email)
print("Mobile:", candidate_phone)
print("Skills:", ", ".join(skills) if skills else "None found")
print("Experience:", years, "years")
print("Shortlisted:", "Yes" if years >= 2 and any(s.lower()
    == "python" for s in skills) else "No")
```

Output Screenshot



Result

The program successfully extracts resume details using regular expressions and shortlists the candidate because the candidate has 3 years of experience and Python skill.

2. Product Search

Aim

To build a Python product search system using Regular Expressions that supports exact, prefix, suffix, partial, case-insensitive, and price-based searches, and reports the matching products for each search.

Algorithm

1. Start and create a sample list of product names with prices.
2. Read each search pattern and search type.
3. Use regular expressions with the appropriate anchors or partial matching.
4. Use re.IGNORECASE for case-insensitive searching.
5. For price-based searching, select products whose price is within the specified limit.
6. Display matching products and the number of matches; stop.

Python Code

```
import re products

= [
   ("Python Programming Book", 450),
   ("Wireless Mouse", 650),
   ("Python Practice Workbook", 300), ("USB
   Keyboard", 550),
   ("Laptop Stand", 800)
]

def search_products(pattern, mode): matches =
   []
   for name, price in products: if
      mode == "exact":
         found = re.fullmatch(pattern, name, re.IGNORECASE) elif
      mode == "prefix":
         found = re.match(pattern, name, re.IGNORECASE) elif
      mode == "suffix":
         found = re.search(pattern + r"$", name, re.IGNORECASE) else:
      # partial or case-insensitive search
         found = re.search(pattern, name, re.IGNORECASE) if
      found:
         matches.append((name, price))
   return matches

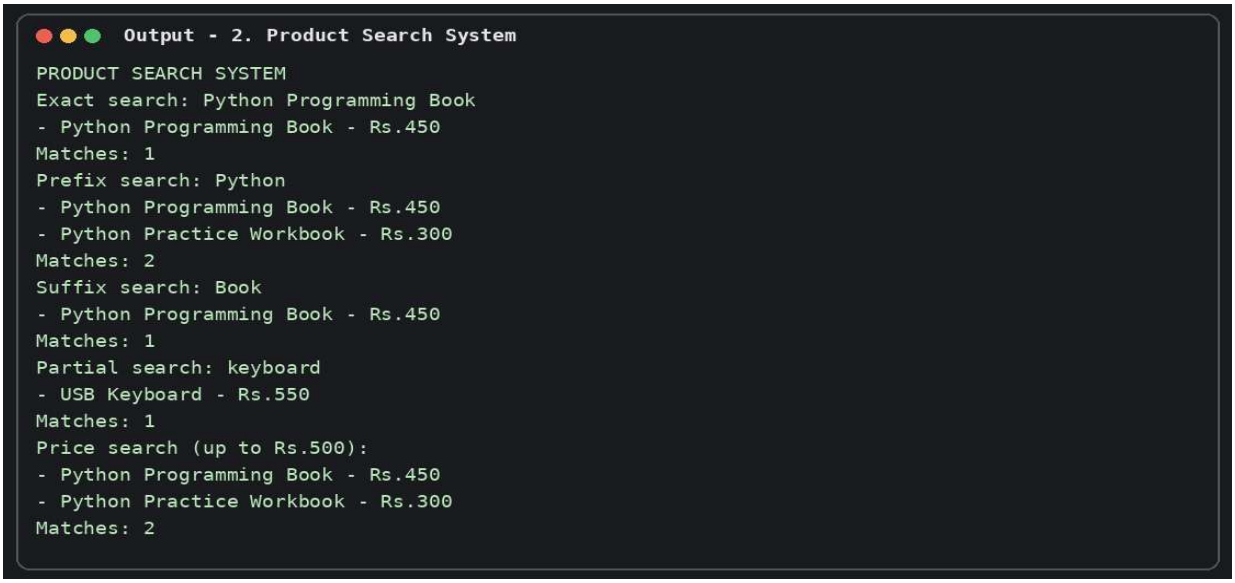
print("PRODUCT SEARCH SYSTEM")
for query, mode in [("Python Programming Book", "exact"),

                    ("Python", "prefix"),
                    ("Book", "suffix"),
                    ("keyboard", "partial")]: results =
```

```
    search_products(query, mode)
    print(f"\\n{mode.title()} search: {query}") for
    name, price in results:
        print(f"- {name} - Rs.{price}")
    print("Matches:", len(results))

limit = 500
cheap = [(n, p) for n, p in products if p <= limit]
print(f"\\nPrice search (up to Rs.{limit}):")
for name, price in cheap:
    print(f"- {name} - Rs.{price}")
print("Matches:", len(cheap))
```

Output Screenshot



Result

The program successfully performs exact, prefix, suffix, partial, case-insensitive, and price-based product searches and displays matching products with counts.

3. University Registration System

Aim

To validate student registration details using Regular Expressions, including register number, email address, course code, semester information, and mobile number, then display validation messages and the final registration status.

Algorithm

1. Start and define sample student registration details.
2. Validate the register number using a regular expression.
3. Validate the email address and 10-digit mobile number.
4. Validate the course code and semester format.
5. Display a success or error message for every field.
6. If all fields are valid, display registration successful; otherwise display registration failed.

Python Code

```
import re student =

{
   "register_no": "23CSE1045", "email":
   "student@example.com",
   "course_code": "CSE301",
   "semester": "Semester 5",
   "mobile": "9876543210"
}

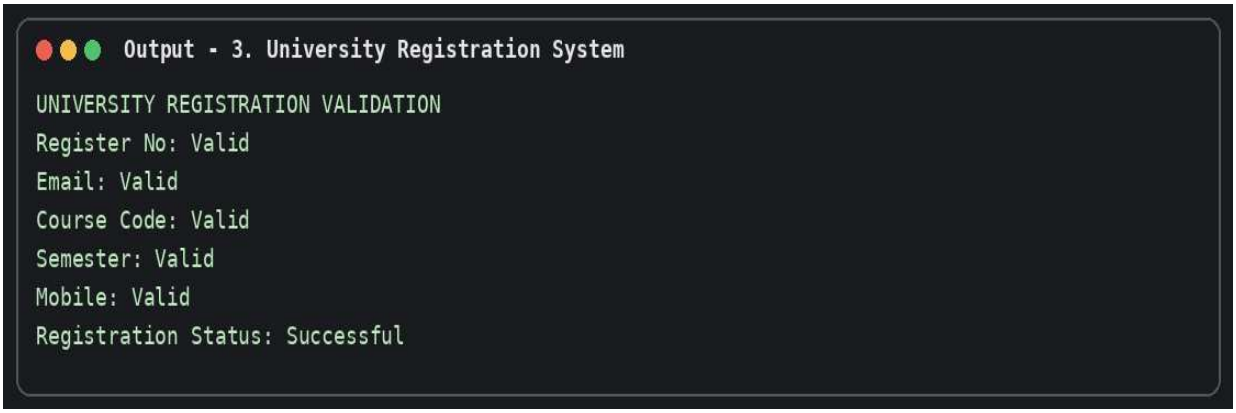
rules = {
   "register_no": r"\d{2}[A-Z]{3}\d{4}",
   "email": r"[\w.-]+@[\w.-]+\.\w+",
   "course_code": r"[A-Z]{3}\d{3}",
   "semester": r"Semester\s[1-8]",
   "mobile": r"[6-9]\d{9}"
}

print("UNIVERSITY REGISTRATION VALIDATION")
all_valid = True

for field, pattern in rules.items():
   valid = bool(re.fullmatch(pattern, student[field]))
   print(f"{field.replace('_', ' ').title()}:",
        "Valid" if valid else "Invalid") if
   not valid:
      all_valid = False

print("Registration Status:",
     "Successful" if all_valid else "Failed")
```

Output Screenshot



Result

The program successfully validates the student registration fields and displays a successful registration status because all sample values match the validation rules.